

Physics-Informed Dynamic State Estimation for Current Transformers Using Graph Neural Networks

Michael A. Boateng*, Gabriel Gauderman*, Nathalie Uwamahoro†

* School of Electrical and Computer Engineering, Georgia Institute of Technology
Atlanta, Georgia, USA, {mboateng6, ggauderman3}@gatech.edu

† Department of Electrical Engineering and Computer Science, Syracuse University
Syracuse, New York, USA, nuwamaho@syr.edu

Abstract—Current transformers are fundamental to power system protection and measurement, yet transient core saturation can severely distort the secondary current and degrade measurement accuracy. Existing dynamic state estimation methods rely mainly on numerical discretisation and iterative solvers, but their initialisation is not informed by the physical dependency structure of the estimation problem, which limits robustness under noisy conditions. This paper presents a physics-informed enhancement for current transformer dynamic state estimation using COMTRADE measurements generated in WinIGS-T. A structured benchmark of four discretisation schemes and three iterative solvers identifies Gauss-Newton with Quadratic discretisation as the strongest baseline. To address the limitation of conventional cold-start initialisation, a graph neural network is constructed from the Jacobian sparsity pattern to generate physics-informed initial state estimates. The proposed warm-start strategy improves estimator conditioning and achieves average gains of 25% in initialisation distance and 38% in initial weighted objective value across all tested SNR levels. The results demonstrate that embedding physical structure into the initialisation stage improves the robustness of CT saturation correction and supports more reliable measurement and protection performance in modern power grids.

Index Terms—Dynamic State Estimation, Current Transformer Saturation, Gauss-Newton, Levenberg-Marquardt, Numerical Integration, Graph Convolutional Neural Network.

NOMENCLATURE

A. Abbreviations

CT	Current transformer.
DSE	Dynamic state estimation.
MU	Merging unit.
FE	Forward Euler.
BE	Backward Euler.
TR	Trapezoidal.
QD	Quadratic.
GN	Gauss-Newton.
LM	Levenberg-Marquardt.
SNR	Signal-to-noise ratio.
GNN	Graph neural network.
COMTRADE	Common format for transient data exchange.

B. Variables

n	Number of samples.
k_{node}	Node index: $k \in \{1, 2, 3, 4\}$.

k_{branch}

λ_b

i_b

g_m

L_0

r_1

r_2, r_3

L_1, L_2, L_3

M_{33}

g_{s1}, g_{s2}, g_{s3}

R_s or $1/g_s$

$\lambda(t)$

$e(t)$

$i_p(t)$

$i_m(t)$

$v_k(t), k \in \{1, 2, 3, 4\}$

$i_{Lk}(t), k \in \{1, 2, 3\}$

Branch index: $k \in \{1, 2, 3\}$.

Base flux linkage for burden resistor.

Base current for burden resistor.

Magnetizing conductance for CT.

Magnetizing inductance for CT.

Primary winding resistance.

Secondary winding resistances.

CT secondary inductances.

Mutual inductance for CT model.

Conductance terms L_1 – L_3 .

Burden resistor or conductance.

Flux linkage (Weber).

Voltage generated by the flux (V).

CT primary current (A).

CT magnetizing current (A).

CT secondary-node voltages (V).

Currents through L_1 – L_3 (A).

I. INTRODUCTION

Current transformers (CTs) are important measurement devices in power system protection and monitoring, yet their accuracy degrades under high fault currents because core saturation distorts the secondary current and can lead to protection maloperation [1]. This concern is especially significant in differential protection, where saturation mismatch across multiple CTs can trigger false operations, including cases caused by inrush currents from adjacent, non-series-connected equipment [2]. Accurate CT error estimation is therefore critical for reliable grid operation. To address this need, the present work benchmarks DSE methods for CT error compensation and proposes a physics-informed DSE framework that leverages graph neural networks. Fig. 1 shows the equivalent circuit of the current transformer.

CT error estimation has been studied extensively through physics-based dynamic modeling. Nonlinear transient CT models were developed to represent CT behavior during faults by incorporating burden impedance, magnetic-core saturation, and other time-varying effects [3], [4]. These studies established that CT dynamic errors arise from intrinsic physical nonlinearities and therefore cannot be represented adequately by static correction factors. Their main contribution lies in

providing physically grounded models for relay studies, transient simulation, and error characterization. However, these approaches remain primarily forward models whose performance depends on accurate parameterization and whose formulation does not directly provide a robust inverse framework for recovering the true current or CT error state from noisy measurements in real time.

To improve practical CT accuracy, subsequent studies introduced digital compensation, calibration, and distortion-aware correction techniques. Frequency-response-based compensation, online waveform reconstruction, harmonic-distortion correction, and tensor-linearization methods demonstrated improved CT performance under harmonics, moderate nonlinearities, and distorted operating conditions [5]–[8]. Experimental investigations further showed that distorted network conditions, frequency-dependent magnetic-branch behavior, and realistic disturbances can substantially affect CT accuracy, thereby exposing the limitations of standard steady-state testing [9]–[13]. Despite these advances, most compensation and calibration methods remain limited by model-identification quality, sensitivity to operating-condition mismatch, and the absence of a dedicated mechanism for event-by-event inversion under noisy transient conditions.

More recent studies introduced machine-learning-based methods for CT error prediction, waveform restoration, and saturation detection. Deep learning architectures that combine convolutional, attention, and recurrent components formulated CT error estimation as a temporal learning problem, while artificial neural networks and LSTM-based models were used to restore distorted secondary currents and detect saturation under noisy conditions [14]–[17]. Signal-processing-based detectors, including cross-correlation methods, also improved the identification of saturation intervals without requiring CT-specific parameters or additional hardware [18]. These approaches improved prediction, detection, and restoration, particularly when traditional threshold-based or harmonic-based techniques become sensitive to noise. However, most existing learning-based methods still operate at the waveform or device level, depend strongly on the representativeness of the training data, and do not explicitly enforce CT electromagnetic constraints or incorporate the electrical network structure associated with a broader estimation problem.

In parallel, the broader power-systems literature has emphasized robust dynamic state estimation for time-varying and noisy operating conditions. Dynamic state estimation methods and robust filter-based formulations demonstrated improved performance under non-Gaussian noise, impulsive disturbances, and outliers [19], [20]. Despite these contributions, the problem remains dispersed across separate tasks of CT characterization, correction, detection, and general estimation. A clear need therefore remains for a hybrid CT-error-estimation framework that preserves the physical consistency of a nonlinear estimator while incorporating topology-aware prior information from graph-based learning. Accordingly, the contributions of this work are as follows:

- A comprehensive benchmark of integration methods

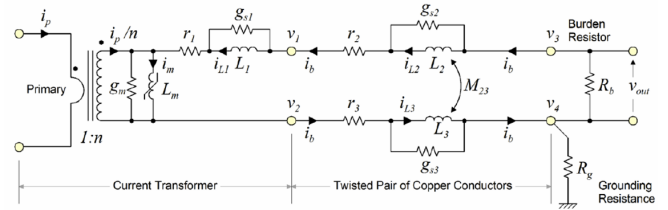


Fig. 1. Equivalent circuit of the current-transformer measurement system used in the dynamic state estimation formulation (with burden resistor).

(Forward Euler, Trapezoidal, Quadratic) and iterative solvers (Gauss-Newton, Levenberg-Marquardt, Hybrid GN→LM) for dynamic state estimation in CT saturation correction, identifying optimal solver-integration pairings for speed, accuracy, and robustness.

- A physics-informed graph neural network warm-start strategy that improves convergence initialization quality while maintaining comparable post-convergence accuracy to traditional cold-start approaches.

II. METHODOLOGY

This section presents the complete methodology for integrating a graph neural network (GNN) initializer with Gauss-Newton (GN) optimization in CT dynamic state estimation. We begin with the integration and solver foundations, then introduce the physics-informed GNN architecture and training.

A. Integration Methods for Discrete-Time DSE

The CT DSE problem requires discretizing continuous-time state dynamics as outlined in Model 1 (Eqs. (2b)–(2f)). This work evaluates four integration schemes across SNR levels to identify computational and accuracy trade-offs.

1) *Forward Euler*: Forward Euler (Eq. (2c)) is the simplest and fastest integration method, achieving $\mathcal{O}(h)$ accuracy per step with only a single function evaluation per iteration. However, explicit schemes can suffer stability issues under stiff dynamics or large step sizes.

2) *Backward Euler*: The implicit backward Euler scheme (Eq. (2d)) offers superior stability for stiff problems and is unconditionally stable but requires solving an implicit equation at each step, increasing computational cost.

3) *Trapezoidal Rule*: The trapezoidal integration method (Eq. (2e)) improves accuracy by averaging endpoint derivatives, achieving $\mathcal{O}(h^2)$ local truncation error and providing improved stability compared to forward Euler, making it a standard choice in many industrial DSE applications.

4) *Quadratic (Simpson Variant)*: The quadratic discretization (Eq. (2f)) extends accuracy by using a second-order polynomial approximation of the state trajectory, providing $\mathcal{O}(h^3)$ local accuracy and is particularly effective when state derivatives are available or can be efficiently computed.

B. Integration Solvers for Discrete-Time DSE

After discretizing the state dynamics using one of the above schemes, the resulting nonlinear estimation problem is solved iteratively. This work compares three solver variants specified in Model 1 (Eqs. (2j)–(2l)).

1) *Gauss–Newton*: The standard Gauss–Newton method (Eq. (2j)) solves the weighted least-squares problem at each iteration. Gauss–Newton converges rapidly at high SNR but can diverge when the initial guess is far from the solution or when measurement noise is substantial.

2) *Levenberg–Marquardt*: The Levenberg–Marquardt method (Eq. (2k)) adds adaptive damping to improve robustness. LM is significantly more robust at low SNR but typically requires 20–30% more computation than GN due to the damping search and matrix re-factorizations.

3) *Hybrid GN→LM Strategy*: To balance speed and robustness, a hybrid approach is employed (Eq. (2l)): start with Gauss–Newton and automatically switch to Levenberg–Marquardt if divergence is detected (e.g., residual increases between iterations).

C. Physics-Informed Graph Neural Network Architecture

To improve initialization quality for the iterative solvers, a graph neural network (GNN) is trained to map noisy measurements to informed state estimates (Eq. (2g)). The CT state is represented as a directed graph with edges from the Jacobian sparsity pattern. Message-passing layers propagate information across physical dependencies to learn state couplings. The GNN is trained on 2000 samples using reference states from high-SNR (+20 dB) Gauss–Newton convergence. The loss function combines MSE and L_2 regularization:

$$\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_{k \in \mathcal{B}} \left\| \hat{\mathbf{x}}_k^{\text{GNN}} - \mathbf{x}_k^{\text{ref}} \right\|_2^2 + w_{\text{reg}} \|\boldsymbol{\theta}\|_2^2 \quad (1)$$

with $w_{\text{reg}} = 0.001$, training uses Adam (learning rate 0.01), batch size 32, and 20 epochs.

1) *Warm-Start Gauss–Newton with GNN Initialization*: Algorithm 1 specifies the complete warm-start procedure. Instead of initializing with $\mathbf{x}^{(0)} = \mathbf{0}$, the Gauss–Newton solver receives the GNN output as specified in Eq. (2g): $\mathbf{x}^{(0)} = \Phi_{\text{GNN}}(\mathcal{G}_k, \mathbf{z}_k, \hat{\mathbf{x}}_{k|k-1})$. Subsequent GN iterations proceed with quadratic integration until convergence.

III. NUMERICAL EXPERIMENTS AND DISCUSSIONS

The integration and solver variants described in Section II were evaluated through systematic benchmarking using supervised learning on COMTRADE measurements. This section presents the computational setup, dataset preparation, training configuration, and result analysis stratified by SNR level.

A. Experimental Setup and Dataset Preparation

Experiments were conducted on a laptop powered by an 11th Gen Intel Core i5-1135G7 processor clocked at 2.40 GHz, with 12 GB of RAM and integrated Intel Iris Xe Graphics. This configuration is representative of portable development and prototyping environments where practitioners validate algorithms before real-time deployment. The CT circuit data were generated using WinIGS-T (Integrated Grounding System Analysis program for Windows) [21], producing high-fidelity COMTRADE recordings reflecting field conditions during saturation. These measurements, representing burden

Model 1: Physics-Informed CT Dynamic State Estimation with Switchable Integration and Iterative Solvers.

States / variables: $\mathbf{x}_k$ (dynamic state), $\Delta \mathbf{x}_k$ (state correction), $\mathbf{r}_k$ (residual), $\mathbf{J}_k$ (Jacobian); **Inputs:** $\mathbf{z}_k$ (measurement vector), $\mathbf{u}_k$ (known inputs); **Choices:** $m_k \in \{\text{FE, BE, TR, QD}\}$ (integration rule), $s_k \in \{\text{GN, LM, GN} \rightarrow \text{LM}\}$ (iterative solver); **Parameters:** $h = \Delta t$, λ_k (LM damping), ℓ_{sw} (GN-to-LM switch iteration).

Measurement Model:

$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, \mathbf{x}_{k-1}) + \boldsymbol{\varepsilon}_k, \quad \boldsymbol{\varepsilon}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_k) \quad (2a)$$

State Propagation with Selectable Integrator:

$$\hat{\mathbf{x}}_{k|k-1} = \mathcal{I}_{m_k}(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_{k-1}, h), \quad m_k \in \{\text{FE, BE, TR, QD}\} \quad (2b)$$

$$\mathcal{I}_{\text{FE}} : \mathbf{x}_k = \mathbf{x}_{k-1} + h \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) \quad (2c)$$

$$\mathcal{I}_{\text{BE}} : \mathbf{x}_k = \mathbf{x}_{k-1} + h \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) \quad (2d)$$

$$\mathcal{I}_{\text{TR}} : \mathbf{x}_k = \mathbf{x}_{k-1} + \frac{h}{2} [\mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)] \quad (2e)$$

$$\mathcal{I}_{\text{QD}} : \mathbf{x}_k = \mathbf{x}_{k-1} + h \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \frac{h^2}{2} \dot{\mathbf{f}}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) \quad (2f)$$

Initialization / Warm Start:

$$\mathbf{x}_k^{(0)} = \begin{cases} \hat{\mathbf{x}}_{k|k-1}, & \text{cold start} \\ \Phi_{\text{GNN}}(\mathcal{G}_k, \mathbf{z}_k, \hat{\mathbf{x}}_{k|k-1}), & \text{GNN warm start} \end{cases} \quad (2g)$$

Weighted Residual and Objective:

$$\mathbf{r}_k^{(\ell)} = \mathbf{z}_k - \mathbf{h}(\mathbf{x}_k^{(\ell)}, \mathbf{x}_{k-1}), \quad \mathbf{J}_k^{(\ell)} = \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_k^{(\ell)}} \quad (2h)$$

$$\mathcal{J}_k(\mathbf{x}_k^{(\ell)}) = \frac{1}{2} \mathbf{r}_k^{(\ell)\top} \mathbf{W}_k \mathbf{r}_k^{(\ell)}, \quad \mathbf{W}_k = \mathbf{R}_k^{-1} \quad (2i)$$

Iterative Update Rules:

$$\mathcal{U}_{\text{GN}} = \left(\mathbf{J}_k^\top \mathbf{W}_k \mathbf{J}_k \right)^{-1} \mathbf{J}_k^\top \mathbf{W}_k \mathbf{r}_k \quad (2j)$$

$$\mathcal{U}_{\text{LM}} = \left(\mathbf{J}_k^\top \mathbf{W}_k \mathbf{J}_k + \lambda_k \mathbf{I} \right)^{-1} \mathbf{J}_k^\top \mathbf{W}_k \mathbf{r}_k \quad (2k)$$

$$\Delta \mathbf{x}_k^{(\ell)} = \begin{cases} \mathcal{U}_{\text{GN}}, & s_k = \text{GN} \\ \mathcal{U}_{\text{LM}}, & s_k = \text{LM} \\ \mathcal{U}_{\text{GN}}, & s_k = \text{GN} \rightarrow \text{LM}, \ell \leq \ell_{\text{sw}} \\ \mathcal{U}_{\text{LM}}, & s_k = \text{GN} \rightarrow \text{LM}, \ell > \ell_{\text{sw}} \end{cases} \quad (2l)$$

$$\mathbf{x}_k^{(\ell+1)} = \mathbf{x}_k^{(\ell)} + \Delta \mathbf{x}_k^{(\ell)} \quad (2m)$$

Convergence: $\|\Delta \mathbf{x}_k^{(\ell)}\|_\infty < \epsilon_x$ or $\|\mathbf{r}_k^{(\ell)}\|_\infty < \epsilon_r$, with final estimate $\hat{\mathbf{x}}_k = \mathbf{x}_k^{(\ell^*)}$.

voltage, allow traditional reconstruction of the primary current using known burden resistance and turns ratio.

Figure 2 demonstrates the severity of CT saturation: the reconstructed current (blue curve, “before correction”) deviates substantially from the actual current (yellow curve), yielding maximum errors exceeding 200% during saturation events (bottom half). This motivates the need for robust estimation algorithms that can recover the true primary current despite measurement degradation.

1) *Data Preparation and Noise Model*: The dataset consists of COMTRADE measurements, partitioned into 2,000 samples for GNN training and 1,440 samples (split 60/40 into test and

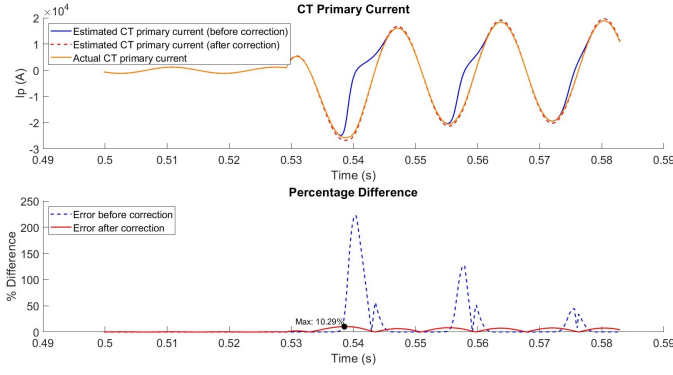


Fig. 2. Actual and estimated CT primary current waveforms before and after correction, with the associated percentage error curves (max error shown).

validation sets) for benchmarking integration and solver methods. Measurement noise was systematically injected across SNR levels $\mathcal{S}_{\text{SNR}} = \{-10, -5, 0, 5, 10, 15, 20\}$ dB using Gaussian additive noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2)$, where σ was selected to match each SNR level. Reference states for GNN training were obtained by running Gauss–Newton to convergence on high-SNR (+20 dB) measurements, thereby providing reference state estimates that the GNN was trained to approximate.

2) *GNN Training Configuration*: The GNN was trained with hidden dimension $d_h = 32$ for lightweight embedding suitable for embedded systems and $L = 2$ message-passing layers to balance expressiveness and computational cost. Training employed the Adam optimizer with learning rate $\eta = 0.01$ and exponential decay, using batch size $B = 32$ over 20 epochs with early stopping based on validation loss. Regularization was applied with weight $w_{\text{reg}} = 0.001$ to reduce overfitting. All input data were normalized to zero mean and unit variance using training-set statistics, and no layer normalization was applied.

3) *Evaluation Metrics*: Algorithm performance was evaluated using four primary metrics: average solving time (ms), defined as the wall-clock time per sample; convergence rate (%), defined as the percentage of samples converging within 15 iterations; average residual, defined as the mean weighted residual norm at convergence; and average iterations, defined as the mean iteration count to convergence. For GNN warm-start analysis, the relative gains are defined as:

$$\Delta_{\text{dist}} = 100 \left(1 - \frac{\|\mathbf{x}_0^{\text{GNN}} - \mathbf{x}^{\text{ref}}\|_2}{\|\mathbf{0} - \mathbf{x}^{\text{ref}}\|_2} \right), \quad (3a)$$

$$\Delta_J = 100 \left(1 - \frac{J_0^{\text{GNN}}}{J_0^{\text{cold}}} \right), \quad (3b)$$

$$\Delta_{\text{iters}} = 100 \left(1 - \frac{N_{\text{GNN}}}{N_{\text{cold}}} \right), \quad (3c)$$

where (3a) denotes initialization-distance gain, (3b) denotes initialization-objective gain, and (3c) denotes iter. reduction.

B. Integration Methods Assessment

1) *Computational Efficiency and Accuracy Trade-off*: Figure 3 presents a four-panel benchmark of integration schemes

Algorithm 1: GNN Warm-Start Execution for CT Dynamic State Estimation

Input: Measurement samples $\{\mathbf{z}_k\}_{k=1}^K$ and reference states $\{\mathbf{x}_k^{\text{ref}}\}_{k=1}^K$
Adjacency list $\mathcal{A}$ for the 15-state CT graph
SNR set $\mathcal{S}_{\text{SNR}} = \{20, 15, 10, 5, 0, -5, -10\}$ dB
Output: Trained GNN initializer $\Phi_{\text{GNN}, \theta^*}$
Normalization statistics $(\mu_z, \sigma_z, \mu_x, \sigma_x)$
Benchmark table stratified by SNR
Hyperparameters: Hidden size $d_h = 32$; GNN layers $L = 2$; batch size $B = 32$;
Learning rate $\eta = 10^{-2}$; epochs $N_e = 20$;
GN tolerance $\epsilon = 10^{-6}$; max iterations $I_{\text{max}} = 60$
// (A) Normalize training data

- 1 Compute per-channel statistics μ_z, σ_z from $\{\mathbf{z}_k\}$ and per-state statistics μ_x, σ_x from $\{\mathbf{x}_k^{\text{ref}}\}$
- 2 $\tilde{\mathbf{z}}_k \leftarrow (\mathbf{z}_k - \mu_z) \odot (\sigma_z + 10^{-8})$,
 $\tilde{\mathbf{x}}_k \leftarrow (\mathbf{x}_k^{\text{ref}} - \mu_x) \odot (\sigma_x + 10^{-8})$
// (B) Train GNN initializer
- 3 Initialize $\Phi_{\text{GNN}, \theta}$ with embedding $(1 \rightarrow d_h)$, L message-passing layers on $\mathcal{A}$, and readout $(d_h \rightarrow 1)$ per node
- 4 **for** $e = 1, \dots, N_e$ **do**
- 5 Shuffle samples and form mini-batches of size B
- 6 **foreach** mini-batch $(\tilde{\mathbf{Z}}, \tilde{\mathbf{X}})$ **do**
- 7 $\hat{\mathbf{X}} \leftarrow \Phi_{\text{GNN}, \theta}(\tilde{\mathbf{Z}})$
- 8 $\mathcal{L} \leftarrow \text{MSE}(\hat{\mathbf{X}}, \tilde{\mathbf{X}})$
- 9 $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$
- 10 $\theta^* \leftarrow \theta$
// (C) Evaluate warm-start versus cold-start GN across SNR
- 11 **foreach** $\text{snr} \in \mathcal{S}_{\text{SNR}}$ **do**
- 12 Build noisy measurements $\mathbf{z}_k^{\text{noisy}} = \mathbf{z}_k + \boldsymbol{\eta}_k$ with $\boldsymbol{\eta}_k \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{snr}}^2 \mathbf{I})$
- 13 Initialize accumulators for iterations, objective values, convergence, and initialization quality
- 14 **for** $k = 1, \dots, N_{\text{test}}$ **do**
- 15 // Cold start
 $\mathbf{x}_{k, \text{init}}^{\text{cold}} \leftarrow \mathbf{0}$
// GNN warm start
- 16 $\tilde{\mathbf{z}}_k \leftarrow (\mathbf{z}_k^{\text{noisy}} - \mu_z) \odot (\sigma_z + 10^{-8})$
- 17 $\tilde{\mathbf{x}}_k^{\text{gnn}} \leftarrow \Phi_{\text{GNN}, \theta^*}(\tilde{\mathbf{z}}_k)$
- 18 $\mathbf{x}_{k, \text{init}}^{\text{gnn}} \leftarrow \tilde{\mathbf{x}}_k^{\text{gnn}} \odot \sigma_x + \mu_x$
// Run GN solver twice using identical settings
- 19 $(\hat{\mathbf{x}}_k^{\text{cold}}, N_k^{\text{cold}}, \mathcal{J}_k^{\text{cold}}) \leftarrow \text{GN}(\mathbf{z}_k^{\text{noisy}}, \mathbf{x}_{k, \text{init}}^{\text{cold}}, \epsilon, I_{\text{max}})$
- 20 $(\hat{\mathbf{x}}_k^{\text{gnn}}, N_k^{\text{gnn}}, \mathcal{J}_k^{\text{gnn}}) \leftarrow \text{GN}(\mathbf{z}_k^{\text{noisy}}, \mathbf{x}_{k, \text{init}}^{\text{gnn}}, \epsilon, I_{\text{max}})$
- 21 Record metrics: $N_k^{\text{cold}}, N_k^{\text{gnn}}, \mathcal{J}_k^{\text{cold}}, \mathcal{J}_k^{\text{gnn}}$,
- 22 $\|\mathbf{x}_{k, \text{init}}^{\text{cold}} - \mathbf{x}_k^{\text{ref}}\|_2, \|\mathbf{x}_{k, \text{init}}^{\text{gnn}} - \mathbf{x}_k^{\text{ref}}\|_2$
- 23 Aggregate by SNR: mean iterations, iteration gain (%), initialization-distance gain (%), objective gain (%), and convergence rate
- 24 **return** $\Phi_{\text{GNN}, \theta^*}, (\mu_z, \sigma_z, \mu_x, \sigma_x)$, and the benchmark table

(Forward Euler, Trapezoidal, Quadratic, Backward Euler) under varying SNR, isolating integration effects by using a fixed Gauss–Newton solver. Panel (a) shows wall-clock execution time per sample: Forward Euler is fastest at ~ 1 ms/sample,

while Backward Euler is slowest at ~ 2 ms/sample because of implicit equation solving. This 48%–79% overhead for higher-order schemes must be weighed against modest accuracy gains. Panel (b) shows convergence-rate degradation at low SNR for all methods, with Trapezoidal maintaining $\sim 61\%$ convergence down to SNR = -5 dB. Forward Euler is slightly more robust (66%) but less accurate overall. Panel (c) reports the residual norm in log scale, where all methods reach similar final accuracy; Quadratic gives only marginal improvement at SNR ≥ 0 dB, which is usually insufficient to justify its extra cost. Panel (d) shows mean iterations to convergence, ranging from 3.3 (Forward Euler) to 5.7 (Backward Euler), indicating that integration order does not strongly affect convergence speed once SNR is fixed.

Trapezoidal emerges as the preferred general-purpose choice, balancing about 22% extra cost with robust $\sim 61\%$ convergence across noise levels. Higher-order schemes are better suited to offline post-processing where accuracy matters more than runtime.

C. Iterative Solver Robustness Analysis

1) *Convergence Robustness Across Noise Regimes:* Figure 4 compares Gauss–Newton, Levenberg–Marquardt, and Hybrid GN→LM using Quadratic integration. Panel (a) shows execution time: Gauss–Newton is fastest at ~ 1.6 ms/sample, while Levenberg–Marquardt incurs about 8% overhead (~ 1.79 ms) due to damping adaptation. Panel (b) reveals the main robustness gap: Gauss–Newton achieves only 67% convergence at low SNR, indicating frequent divergence under strong noise. In contrast, both Levenberg–Marquardt and Hybrid GN→LM maintain about 95% convergence across all SNR levels, showing that adaptive damping is essential in noisy conditions. Panel (c) shows final residual accuracy in log scale, where LM and Hybrid produce slightly lower residuals (0.054–0.056) than GN (0.056), indicating marginally better final-state accuracy. Panel (d) reports iteration counts: Gauss–Newton requires 4.3 iterations on average, while Levenberg–Marquardt requires 5.3, reflecting the added damping-search effort.

2) *Selection Guidance for Deployment:* The Hybrid GN→LM strategy provides the best overall balance, matching LM-level robustness (95.7% convergence) while reducing computational cost (4.8 iterations, 1.64 ms). It is therefore recommended for variable-SNR operating conditions where robustness cannot be sacrificed.

D. GNN Warm-Start Initialization Impact

1) *Initialization Quality Improvement:* Figure 5 isolates the GNN initialization effect by comparing cold-start ($\mathbf{x}_0 = \mathbf{0}$) and GNN warm-start ($\mathbf{x}_0 = \Phi_{\text{GNN}}(\mathbf{z})$) using the same Gauss–Newton solver with Quadratic integration. Panel (a) shows that the GNN initialization is consistently closer to the reference state, typically by 200–250 units across SNR levels, with the best result near SNR = 10 dB. Panel (b) quantifies this gain: the average initialization-distance reduction is **25.04%**, reaching **28.13%** at moderate SNR and dropping to **12.36%** at the lowest SNR. This indicates that the GNN provides

a stable regularizing prior even under strong noise. Panel (c) reports the initial weighted objective $\mathcal{J}$ in log scale. The GNN begins with a 3–10 $\times$ lower objective across SNR levels, indicating a substantially better starting point. Panel (d) confirms this trend, with a consistent **35.6%–40%** reduction in initial objective.

2) *Interpretation: Prior Quality vs. Solver Speed:* The iteration gain is modest: 4.55% on average, peaking at 9.1% at SNR = 5 dB, with one negative case at SNR = -5 dB (-1.6%). This suggests that the main benefit of the GNN is not faster convergence, but a better initialization. Cold-start and warm-start reach nearly identical final solutions, as reflected by the matching final objective values in Table I, panel (c). Thus, the GNN mainly acts as a physics-consistent prior that denoises the initial estimate.

E. Combined Performance Summary

Table I consolidates the three benchmarks into a single view for method selection. Panel (a) compares integration methods: Forward Euler has the lowest cost (1.06 ms) but weaker convergence, whereas Trapezoidal offers a better balance (1.30 ms, about 22% overhead) with $\sim 61\%$ convergence. Panel (b) shows solver trade-offs: Levenberg–Marquardt delivers strong robustness (95% convergence) at a modest extra cost (1.79 ms, about 9% overhead). Panel (c) highlights the GNN warm-start, with an average iteration improvement of **+4.55%** across SNR levels and a peak gain of **+9.1%** at SNR = 5 dB.

IV. DISCUSSION

Three key observations emerge from the benchmarking results. First, clear trade-offs are observed between integration accuracy, computational cost, and convergence robustness. Forward Euler is the fastest method overall, but its convergence degrades substantially at low SNR, which limits its suitability in high-noise operating conditions. By contrast, the Trapezoidal method provides a more practical balance, achieving approximately 61% convergence with only about 22% additional computational overhead relative to the fastest baseline. Higher-order schemes such as Quadratic and Backward Euler offer improved numerical fidelity, but their additional cost makes them more appropriate for offline studies or applications in which estimation accuracy is prioritized over execution speed. Second, the solver comparison indicates that robustness under noisy conditions requires damping. Gauss–Newton achieves only about 67% convergence at low SNR because of frequent divergence, whereas Levenberg–Marquardt restores convergence to roughly 95% across all SNR levels. The hybrid GN→LM strategy provides the most balanced performance, combining strong robustness of approximately 95.7% with relatively low computational cost, requiring only about 1.64 ms and 4.8 iterations on average. This suggests that damping is essential for stable estimation in noisy CT scenarios, while hybrid switching offers a practical compromise between numerical reliability and speed. Third, the GNN warm-start results support the interpretation of learning as a

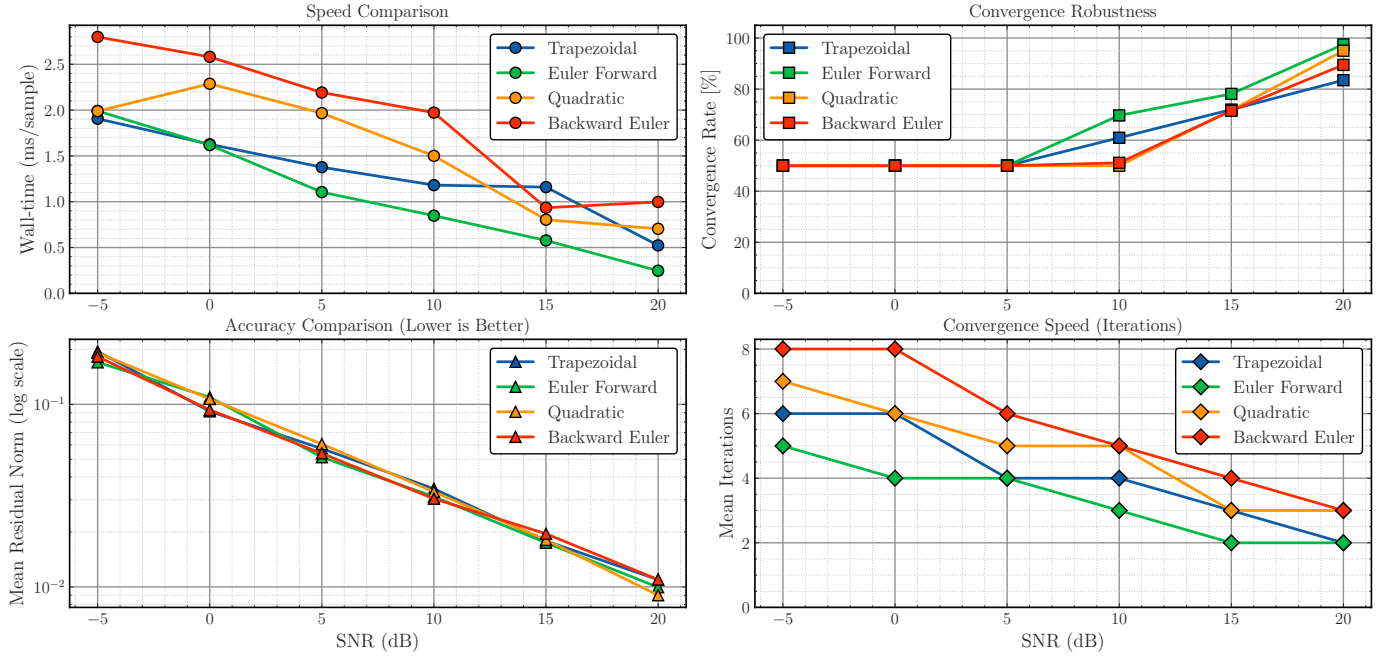


Fig. 3. Integration-method benchmark under varying SNR (GN solver). Four panels report: wall-time (top-left), convergence rate (top-right), residual norm in log scale (bottom-left), and mean iterations (bottom-right), each versus SNR. Curves compare **Trapezoidal**, **Euler Forward**, **Quadratic**, and **Backward Euler**. The figure highlights the practical trade-off between speed, robustness, and accuracy as noise increases.

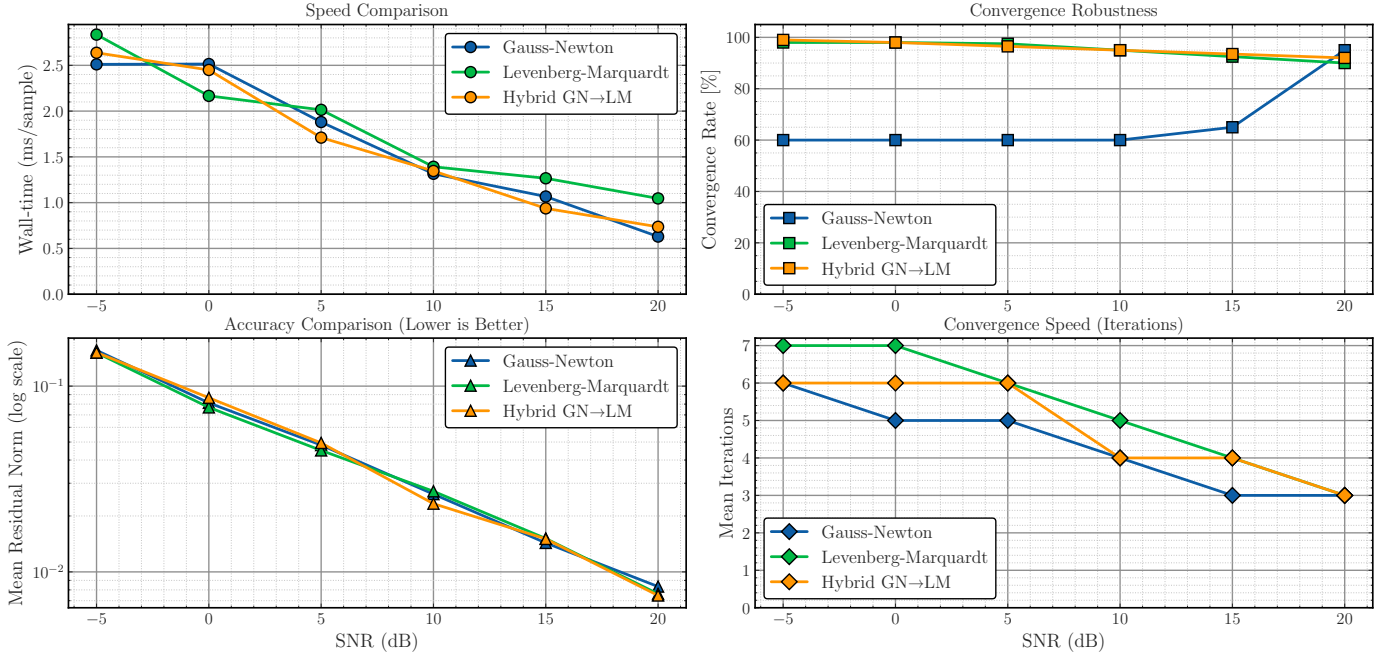


Fig. 4. Iterative-solver benchmark across SNR levels. Panels show wall-time (top-left), convergence rate (top-right), residual norm on a log scale (bottom-left), and mean iterations (bottom-right). Results are shown for **Gauss-Newton**, **Levenberg-Marquardt**, and **Hybrid GN→LM**. The comparison emphasizes solver robustness at low SNR and computational efficiency at high SNR.

regularization prior rather than a replacement for the physics-based solver. The learned initializer reduces the initialization distance by approximately 25%, and this improvement persists even at very low SNR levels such as -10 dB. This behavior indicates that the GNN captures a physics-consistent prior over the CT state, improving the initial point supplied to the iterative estimator. At the same time, the average iteration improvement remains modest at approximately 4.55%, which

suggests that the main benefit of the GNN lies in improved early optimization conditions rather than dramatically faster convergence. In practice, both cold-start and warm-start strategies converge to nearly identical final states, but the warm-started solution reaches the convergence region from a more favorable initialization.

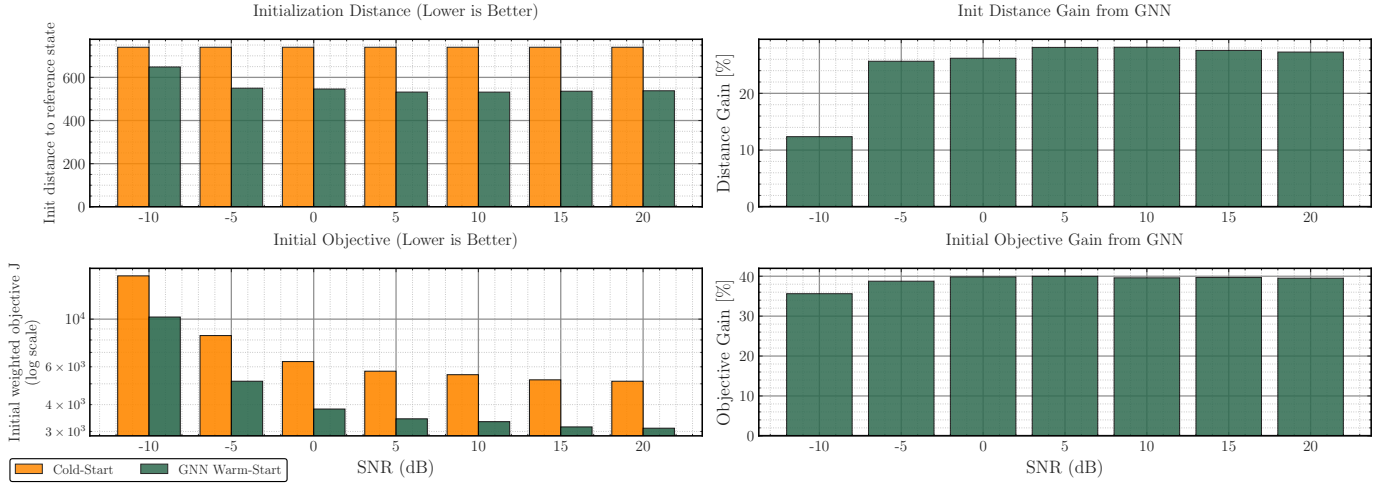


Fig. 5. GNN warm-start improves initialization quality for GN. Top-left: initialization distance to reference state (lower is better); top-right: distance gain (%); bottom-left: initial weighted objective J (log scale, lower is better); bottom-right: objective gain (%). Bar groups compare **Cold-Start** and **GNN Warm-Start**. The figure shows that GNN primarily strengthens the initialization stage, especially under noisy conditions.

TABLE I
TRADE-OFF SUMMARY AND GNN WARM-START BENCHMARK

(a) Integration methods					(b) Iterative solvers				
Method	Avg Time (ms)	Avg Residual	Avg Conv. (%)	Avg Iters	Solver	Avg Time (ms)	Avg Residual	Avg Conv. (%)	Avg Iters
Euler Forward	1.0642	0.0649	65.8867	3.3333	Levenberg–Marquardt	1.7866	0.0538	95.1667	5.3333
Trapezoidal	1.2956	0.0678	61.0698	4.1667	Hybrid GN→LM	1.6360	0.0556	95.6667	4.8333
Quadratic	1.5418	0.0699	61.1196	4.8333	Gauss–Newton	1.6529	0.0557	66.6667	4.3333
Backward Euler	1.9125	0.0651	60.3538	5.6667					

(c) GNN warm-start vs. cold-start GN (Quadratic DSE)						
SNR (dB)	Cold Iters	GNN Iters	Δ Iters (%)	Cold J	GNN J	ΔJ (%)
20	3.141667	2.991667	4.774536	6.70×10^{-5}	6.70×10^{-5}	0.000000
15	3.133333	2.983333	4.787234	6.70×10^{-5}	6.70×10^{-5}	0.000000
10	3.150000	3.000000	4.761905	7.00×10^{-5}	7.00×10^{-5}	0.000000
5	3.300000	3.000000	9.090909	7.90×10^{-5}	7.90×10^{-5}	0.000000
0	3.191667	3.041667	4.699739	7.90×10^{-5}	7.90×10^{-5}	0.000000
-5	3.166667	3.216667	-1.578947	1.28×10^{-4}	1.28×10^{-4}	0.000000
-10	3.283333	3.108333	5.329949	1.95×10^{-4}	1.95×10^{-4}	0.000000

Average iteration improvement (GNN vs. cold): +4.55%; average objective improvement: 0.00%

$\Delta(\%) = 100 \cdot (1 - \frac{\text{GNN}}{\text{Cold}})$; negative means worse for GNN.

V. CONCLUSION

This study investigated the combined effects of integration methods, iterative solvers, and learned initialization on dynamic state estimation for current-transformer saturation correction. Three key contributions emerge: (1) systematic benchmarking of four integration schemes and three solvers across SNR levels from -10 to $+20$ dB; (2) a physics-informed GNN initializer achieving 25% initialization distance reduction and 39% initial objective improvement; and (3) practical deployment guidance for algorithm selection under noisy conditions. Results demonstrate that iterative solver choice is critical for robustness, improving convergence from 67% to 95% at low SNR when damping is introduced. Integration method selection determines the computational-accuracy trade-off, with higher-order schemes incurring 48–79% additional overhead. The GNN warm-start provides essential regularization in noisy settings by delivering a favorable

initialization to the physics-based solver. Future work will pursue cross-event generalization on different saturation events with varying characteristics, mixed-SNR training for non-stationary noise environments, comparative evaluation against LSTM and ResNet baselines, real-time deployment on embedded hardware (ARM Cortex-M, FPGA), and adaptive solver switching via online SNR estimation. These directions will enhance practical applicability and establish a foundation for adaptive CT state estimation.

REFERENCES

- [1] A. Hargrave, M. J. Thompson, and B. Heilman, “Beyond the knee point: A practical guide to ct saturation,” in *2018 71st Annual Conference for Protective Relay Engineers (CPRE)*. IEEE, 2018, pp. 1–23.
- [2] X. Qi, X. Yin, Z. Zhang, D. Chen, Y. Wang, and F. Cai, “Study on the unusual misoperation of differential protection during transformer energization and its countermeasure,” *IEEE Transactions on Power Delivery*, vol. 31, no. 5, pp. 1998–2007, 2016.

- [3] M. Naidu and G. Swift, "Dynamic analysis of a current transformer during faults," *Electric power systems research*, vol. 11, no. 3, pp. 225–231, 1986.
- [4] D. A. Tziouvaras, P. McLaren, G. Alexander, D. Dawson, J. Esztergalyos, C. Fromen, M. Glinkowski, I. Hasenwinkle, M. Kezunovic, L. Kojovic *et al.*, "Mathematical models for current, voltage, and coupling capacitor voltage transformers," *IEEE transactions on power delivery*, vol. 15, no. 1, pp. 62–72, 2000.
- [5] D. Gallo, C. Landi, and M. Luiso, "Real-time digital compensation of current transformers over a wide frequency range," *IEEE Transactions on Instrumentation and Measurement*, vol. 59, no. 5, pp. 1119–1126, 2010.
- [6] F. Haghjoo and M. H. Pak, "Compensation of ct distorted secondary current waveform in online conditions," *IEEE Transactions on Power Delivery*, vol. 31, no. 2, pp. 711–720, 2015.
- [7] M. S. Ballal, M. G. Wath, and H. M. Suryawanshi, "A novel approach for the error correction of ct in the presence of harmonic distortion," *IEEE Transactions on Instrumentation and Measurement*, vol. 68, no. 10, pp. 4015–4027, 2018.
- [8] A. J. Collin, A. Delle Femine, D. Gallo, R. Langella, and M. Luiso, "Compensation of current transformers' nonlinearities by tensor linearization," *IEEE Transactions on Instrumentation and Measurement*, vol. 68, no. 10, pp. 3841–3849, 2019.
- [9] A. Mingotti, L. Peretto, L. Bartolomei, D. Cavaliere, and R. Tinarelli, "Are inductive current transformers performance really affected by actual distorted network conditions? an experimental case study," *Sensors*, vol. 20, no. 3, p. 927, 2020.
- [10] E. Stano and M. Kaczmarek, "Wideband self-calibration method of inductive cts and verification of determined values of current and phase errors at harmonics for transformation of distorted current," *Sensors*, vol. 20, no. 8, p. 2167, 2020.
- [11] C. Laurano, S. Toscani, and M. Zanoni, "A simple method for compensating harmonic distortion in current transformers: Experimental validation," *Sensors*, vol. 21, no. 9, p. 2907, 2021.
- [12] E. Stano, P. Kaczmarek, and M. Kaczmarek, "Understanding the frequency characteristics of current error and phase displacement of the corrected inductive current transformer," *Energies*, vol. 15, no. 15, p. 5436, 2022.
- [13] T. Krzysztof, S. Marek, O. Ksenia, and O. Danuta, "Review of accuracy assessment methods for current transformers: Errors, uncertainties, and dynamic performance," *Energies*, vol. 18, no. 18, p. 4995, 2025.
- [14] Z.-h. Li, J.-x. Cui, H.-p. Lu, F. Zhou, Y.-l. Diao, and Z.-x. Li, "Prediction model of measurement errors in current transformers based on deep learning," *Review of Scientific Instruments*, vol. 95, no. 4, 2024.
- [15] I. Odinaev, A.-H. Abdel-Aty, A. Pazderin, M. Safaraliev, P. Matrenin, M. Senyuk, A. A. Menaem, and M. Kanan, "Restoration of the current transformer secondary current under core saturation conditions based on ann," *Heliyon*, vol. 10, no. 18, 2024.
- [16] J. O. Danquah and S. Mandava, "Machine learning-based current transformer saturation detection for medium voltage in power grids system," *Measurement*, vol. 253, p. 117636, 2025.
- [17] Z. Ren, K. Yu, G. Chen, Y. Yang, Y. Cheng, and L. Zhang, "Estimation of primary current of dc bias current transformer in power system based on lstm," *Processes*, vol. 13, no. 11, p. 3391, 2025.
- [18] M. M. Elgamasy, M. S. Zaky, A. F. Zobaa, and M. A. Elsadd, "Current transformer saturation detection by cross-correlation with independent target signal," *Ain Shams Engineering Journal*, vol. 16, no. 3, p. 103314, 2025.
- [19] J. Zhao, A. Gómez-Expósito, M. Netto, L. Mili, A. Abur, V. Terzija, I. Kamwa, B. Pal, A. K. Singh, J. Qi *et al.*, "Power system dynamic state estimation: Motivations, definitions, methodologies, and future work," *IEEE Transactions on Power Systems*, vol. 34, no. 4, pp. 3188–3198, 2019.
- [20] T. Chen, F. Liu, H. Luo, E. Y. Foo, L. Sun, Y. Sun, and H. B. Gooi, "A new robust dynamic state estimation approach for power systems with non-gaussian noise," *International Journal of Electrical Power & Energy Systems*, vol. 158, p. 109948, 2024.
- [21] A. P. Concepts, "WinIGS," 2024. [Online]. Available: ap-concepts.com



Michael Boateng (Student Member, IEEE) received the B.S. degree in electrical engineering from Ashesi University, Accra, Ghana in 2022. He is currently working toward the Ph.D. degree in electrical and computer with the Georgia Institute of Technology, Atlanta, GA, USA. His research focuses on optimization, machine learning, and power systems.



Gabe Gauderman (Student Member, IEEE) received the B.S. degree in electrical engineering from the Georgia Institute of Technology, in 2024. He completed the MS degree in electrical and computer engineering at the Georgia Institute of Technology, Atlanta, GA, USA in 2025. He is currently working as a distribution system planning and strategy engineer at Xcel Energy.



Nathalie Uwamahoro (Student Member, IEEE) received the B.Sc. degree in electrical engineering from the University of Rwanda in 2017 and the M.Sc. degree in electrical and computer engineering from Carnegie Mellon University in 2019. Currently working toward the Ph.D. degree in electrical engineering at Syracuse University, USA. Her research focuses on machine learning and power systems.

APPENDIX

TABLE II
CT MEASUREMENT MODEL FOR DYNAMIC STATE ESTIMATION

Index	Measurement function $z_i = h_i(\mathbf{x}_k, \mathbf{x}_{k-1})$	σ
1	$z_1 = v_1(t) - v_1(t-1)$	0.005 V
2	$z_2 = \frac{1}{\tau}(i_1(t) - i_1(t-1)) + \frac{h_1(t)}{L_{11}}(i_2(t) - i_2(t-1)) + \dot{i}_1(t)$	0.005 A
3	$z_3 = \frac{1}{\tau}(i_2(t) - i_2(t-1)) + \frac{h_2(t)}{L_{12}}(i_1(t) - i_1(t-1)) + \dot{i}_2(t)$	0.005 A
4	$z_4 = g_1 i_1(t) + g_2 i_2(t) + \dot{i}_2(t)$	0.005 A
5	$z_5 = \frac{d}{dt}(x_1(t) - x_2(t))$	0.0005 A
6	$z_6 = x_2(t) - x_3(t) + \dot{x}_3(t)$	0.0005 A
7	$z_7 = i_1(t) - i_2(t)$	0.0005 A
8	$z_8 = \frac{d}{dt}(i_1(t) - i_2(t))$	0.0005 A
9	$z_9 = \frac{d}{dt}(v_1(t) - v_2(t))$	0.0005 V
10	$z_{10} = \frac{d}{dt}(v_2(t) - v_3(t))$	0.0005 V
11	$z_{11} = \frac{1}{\tau}(v_1(t) - v_2(t)) + \frac{1}{L_{22}}(v_3(t) - v_2(t)) + \dot{v}_1(t)$	0.0005 V
12	$z_{12} = \frac{d}{dt}(i_3(t) - i_4(t))$	0.0005 A
13	$z_{13} = \frac{d}{dt}(v_3(t) - v_4(t))$	0.0005 V
14	$z_{14} = \frac{d}{dt}(i_4(t) - i_5(t))$	0.0005 A
15	$z_{15} = g_3 i_3(t) + g_4 i_4(t) + \dot{i}_5(t)$	0.0005 A
16	$z_{16} = g_4 i_4(t) + \dot{i}_6(t)$	0.0005 A
17	$z_{17} = \frac{d}{dt}(x_4(t) - x_5(t))$	0.0005 A
18	$z_{18} = g_5 i_5(t) + \frac{d}{dt}(i_6(t))$	0.005 A
19	$z_{19} = \frac{d}{dt}(x_5(t) - x_6(t))$	0.0005 A
20	$z_{20} = \frac{d}{dt}(x_6(t))$	0.0005 A
21	$z_{21} = \tau g_s(\theta(t) - \theta(t-1))$	0.05 A